Enhancing the Power of Polyhedral-Based Optimizations with Coordinate Descent

GAURAV VERMA, Stony Brook University, USA
MICHAEL CANESCHE, Cadence, Brazil
BARBARA CHAPMAN, Stony Brook University, USA
FERNANDO MAGNO QUINTÃO PEREIRA, Federal University of Minas Gerais, Brazil

Polyhedral optimizations utilize a geometric model of loop iterations to enable transformations like tiling, fusion, and fission. Tools based on polyhedral optimizations, such as Pluto or Polly, can generate highly efficient code through purely static techniques. However, because they are entirely static, these tools struggle to account for the characteristics of the target architecture. As a result, modern kernel schedulers often avoid the polyhedral model, favoring profiling-based techniques that explore the optimization space dynamically. In this work, we address this limitation by integrating a dynamic post-optimization tuner based on coordinate descent into Pluto, a polyhedra-based optimizer. Our approach uses polyhedral-based static analyses to identify a promising kernel "shape": the order and structure of the loops that define the kernel. We then apply coordinate descent to fine-tune the parameters of these loops, such as tiling sizes and unrolling factors. By incorporating this technique into Pluto, we demonstrate that the resulting kernels outperform those produced by fully static optimizers like Clang -O3 or IOOpt, as well as profile-guided techniques like Apache TVM.

CCS Concepts: • **Software and its engineering → Compilers**.

Additional Key Words and Phrases: Kernel, Polyhedron, Coordinate Descent

ACM Reference Format:
Gaurav Verma, Michael Canesche, Barbara Chapman, and Fernando Magno Quintão Pereira. 2026. Enhancing the Power of Polyhedral-Based Optimizations with Coordinate Descent. 1, 1 (August 2026), 23 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The polyhedral model is a mathematical framework that optimizes loops in programs by representing their iteration space as a polyhedron (a geometric shape in multi-dimensional space)[1]. The theory of polyhedra is important because it allows the systematic representation and optimization of loop-intensive code. By modeling loop iterations as points in a high-dimensional geometric space, polyhedra enable precise analyses of data dependencies, allowing compilers to safely apply complex transformations such as loop interchange, tiling, and fusion to improve cache utilization and parallelism [1, 10, 27, 36, 41, 55, 62, 66].

[1]Notions of iteration and data space are standard in the compiler literature. Such concepts appeared independently in several works during the 80s and 90s [2, 20, 21, 23, 30, 31, 42, 43, 46–48, 52, 60, 60, 61], eventually leading to the theory known today as the *Polyhedral Model*.

Authors' addresses: Gaurav Verma, Stony Brook University, USA, gaurav.verma@stonybrook.edu; Michael Canesche, Cadence, Brazil, canesche@cadence.com; Barbara Chapman, Stony Brook University, USA, barbara.chapman@stonybrook.edu; Fernando Magno Quintão Pereira, Federal University of Minas Gerais, Brazil, fernando@dcc.ufmg.br.

ACM XXXX-XXXX/2026/8-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Polyhedra: A Golden Age Deferred? Given the tremendous popularity of deep-learning models—and the fact that these models are primarily implemented as nests of loops—one might expect the polyhedral model to be enjoying its golden age. However, modern kernel optimizers like Ansor [65], Atlas [59], AutoTVM [15], cuDNN [17], Halide [44], OpenTuner [3], Spiral [24], TensorComprehensions [57], and Tiramisu [5] often take a different approach. Rather than relying on purely static analyses, they perform an *empirical exploration* of the optimization space. These tools employ techniques such as evolutionary search, gradient-based methods, and machine learning to sample the universe of high-performance kernel implementations.

This empirical approach enables compilers to fine-tune performance for specific hardware platforms by capturing hardware characteristics—such as cache hierarchies, vectorization capabilities, and memory bandwidth—that static models like the polyhedral approach may not fully account for. By collecting performance data of multiple versions of the kernel, and iteratively refining them, these compilers can adapt to the intricacies of the target architecture. This dynamic search process is often referred to as *auto-tuning*. While auto-tuning is more hardware-aware than polyhedral-based optimizations, it requires the costly execution of multiple versions of the same program.

The Contribution of this Work. This paper presents a methodology that combines the best of two worlds: static polyhedral-based optimizations and empirical auto-tuning. We leverage polyhedral tools to identify a promising "shape" for a kernel. In the context of this paper, the shape of a kernel is defined by the ordering of its loops and memory access patterns. Once this shape is established, we apply a line-search technique—coordinate descent—to fine-tune optimization parameters such as tile sizes and loop unrolling factors. This approach captures the strengths of both static analysis and empirical tuning, offering advantages over both:

Inter-Space: Line search alone is not an efficient auto-tuning strategy. While it can adjust parameters of a given kernel, it cannot alter the kernel's shape, which determines the search space. As demonstrated in previous works [12, 32], a limitation of coordinate descent is its reliance on a well-defined search space. By integrating polyhedral analysis, we overcome this limitation and address a question left open by Canesche et al. [12] and Zhao et al. [63]; namely, how to identify a promising search space for gradient-based auto-tuning methods.

Fast Start: Although auto-tuners like Ansor can explore different kernel shapes, they require a significant number of samples to find promising configurations, as noted by previous research [13, 32, 63]. In contrast, polyhedral analysis determines effective kernel shapes without sampling. Tools like Pluto [10] and Polly [26] provide static loop ordering and tiling, thus improving cache locality and enabling vectorization, right from the start.

Summary of Results. We have implemented the ideas proposed in this paper in Pluto [10][2]. Our optimizer works in two phases: it uses Pluto to obtain a "seed" kernel and then uses coordinate descent to adjust the optimization parameters of this kernel. Coordinate descent yields optimal solutions when searching on convex surfaces. However, even when this property is absent, Section 4 still shows the effectiveness of our optimizer by running it on well-known kernels. Our version of Pluto generates code that outperforms different static optimizers: not only the original implementation of Pluto itself, but also clang -O3 and IOOpt [37]. This version of Pluto can even produce kernels that outperform those produced by AutoTVM [15] while using a fraction of the search time required by this tool. Section 4.7 further demonstrates the generality of the approach, showing that it can be adapted to perform thread block tuning in graphics processing units.

[2]The ideas discussed in this paper should be applicable to other polyhedral tools. We chose Pluto because a recent comparison between polyhedral compilers involving tools such as Polly, PoCC, and PPCG shows that it delivers some of the highest observed speedups [54].

2 LIMITATIONS OF POLYHEDRAL OPTIMIZATIONS

The polyhedral model treats each loop iteration as a point in the loop's *iteration space*. The model uses inequalities to define the bounds and dependencies between these iterations. This combination of bounds and dependencies provides a theory that enables loop transformations such as tiling, fusion, and interchange. Tools that utilize the polyhedral model to optimize programs, such as Pluto [10] or Polly [26], generally work in three phases:

Static analysis: The polyhedral optimizer analyzes the iteration space to determine how loops interact with each other and the data.

Rewriting: The optimizer applies transformations like tiling or loop interchange to improve data locality and parallelism.

Code generation: The transformed code is generated, resulting in optimized kernel execution.

Example 2.1 illustrates these notions of iteration space and dependencies.

Example 2.1. Figure 1 shows a basic implementation of matrix multiplication, $C = A \times B$, written in the C programming language. Each loop forms a dimension of the iteration space. In Figure 1, we can represent these as a 3D space with `i`, `j`, and `k` forming the axes. Each point in this 3D space corresponds to a specific combination of loop indices (`i`, `j`, `k`) for which an operation is performed. The iteration space is bounded by the loop limits. For instance, for `i`, the bounds are $0 \leq$ `i` $<$ `n`. The polyhedral model captures these bounds as inequalities that form a shape in higher-dimensional space—a polyhedron. The polyhedral model also captures dependencies between iterations. In Figure 1, there is no dependency between different iterations of `i` and `j`, but there is a dependence within the `k` loop because `C[i][j]` is updated in every iteration of `k`.

```
01 void matmul(int n, float A[n][n], float B[n][n], float C[n][n]) {
02     for (int i = 0; i < n; i++) {
03         for (int j = 0; j < n; j++) {
04             C[i][j] = 0.0;
05             for (int k = 0; k < n; k++) {
06                 C[i][j] += A[i][k] * B[k][j];
07             }
08         }
09     }
10 }
```

Fig. 1. The implementation of basic matrix multiplication, used to illustrate the effects of polyhedral transformations.

2.1 Polyhedral Transformations

The polyhedral model enables optimizations like loop interchange and tiling. This theory guarantees that these transformations preserve the program's correctness by ensuring that memory dependencies across different loop iterations are respected. Once the dependencies are captured, the model can safely apply transformations like loop interchange (reordering loops to improve data locality) and tiling (breaking loops into smaller blocks to optimize cache usage). Example 2.2 illustrates two of these transformations.

Example 2.2. Figure 2 shows the code that results from applying loop interchange and tiling to the program in Figure 1. Tiling partitions the loops into smaller blocks (tiles) to enhance data locality and improve cache usage. The compiler tiles the `k` and `j` loops because tiling those dimensions

maximizes the reuse of data from A and B within the cache before moving on to the next tile. The compiler also interchanges the loops k and j to improve locality when accessing matrix A. The optimized program runs faster: on a Xeon Max at 1.9GHz, the program in Figure 1, compiled with gcc -O3, with $10^3 \times 10^3$ matrices, takes 1.02 seconds, whereas the program in Figure 2 takes 0.101 seconds.

```
01 #define K 32
02 #define J 32
03
04 void tiled(int n, float A[n][n], float B[n][n], float C[n][n]) {
05     for (int i = 0; i < n; i++) {
06         for (int k = 0; k < n; k += K) {
07             for (int kk = k; kk < k + K && kk < n; kk++) {
08                 float temp = A[i][kk];
09                 for (int j = 0; j < n; j += J) {
10                     for (int jj = j; jj < j + J && jj < n; jj++) {
11                         C[i][jj] += temp * B[kk][jj];
12                     }
13                 }
14             }
15         }
16     }
17 }
```

Fig. 2. Matrix multiplication (see Figure 1) after loop interchange and tiling.

2.2 Parameter Tuning

The optimizations that produced the program in Figure 2 rely on two parameters: K and J, the dimensions of the two tiling windows. The values in Figure 2, $K = J = 32$, are the default dimensions used by Pluto, for instance. Regardless of the target architecture, Pluto will always use the same parameters. However, they are not necessarily the best values for these parameters. Example 2.3 provides empirical evidence to support this statement.

Example 2.3. Figure 3 shows the search space formed by the tiling parameters J and K defined in Figure 2. The origin of this space is $(J = 1, K = 1)$, meaning no tiling. Pluto sets the implementation of the kernel that it produces at $(J = 32, K = 32)$, for a running time of 101 seconds on the Xeon Max. An exhaustive search on the grid $2^0 \leq J \leq 2^9, 2^0 \leq K \leq 2^9$ reveals that the optimal kernel implementation uses $(J = 256, K = 512)$, achieving a running time of 58 seconds: a speedup of 43% over Pluto's default choice of parameters.

Figure 3 shows that the best implementation of the kernel uses parameters that differ from those that Pluto chooses by default. However, the figure also shows another important fact: the region that goes from the origin of the search space to its optimal configuration is *convex*. A convex subregion within the *hypersurface*[3] that a function determines is a subset of the hypersurface where, for any two points within this region, the straight line segment connecting them lies entirely within the subregion, reflecting the property of convexity. In a convex region, any local minimum is guaranteed to be a global minimum. This makes optimization much simpler and more efficient,

[3]A hypersurface is a shape of $n - 1$ dimensions embedded in an n-dimensional space, defined by a function. In Example 2.3, this function maps (J, K) to running time.

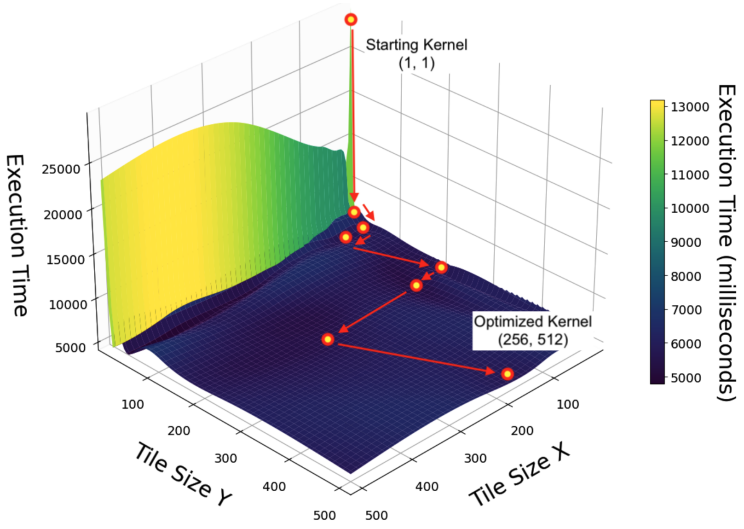

Fig. 3. The search space of the two tiling parameters used in Figure 2.

as there are no local minima "traps" to mislead search algorithms. The observation that the region containing the origin of a kernel's search space and its global optimum is convex is known as the *Droplet Hypothesis* [12]. As Section 4 demonstrates, convexity is not a necessary requirement for the adjustment policy proposed in this paper to work. Nevertheless, whenever this property is present, said adjustment is guaranteed to stabilize on an optimal kernel configuration.

3 COORDINATE DESCENT OVER POLYHEDRA

Coordinate descent is an optimization algorithm that solves problems by iteratively minimizing along one coordinate (or variable) at a time, while holding all other variables fixed. The essence of the algorithm consists of four steps:

Initialization: Start with an initial guess for the solution, and let it be the current kernel candidate (see Section 3.1).

Iterative Updates: For each iteration, find the neighbor of the current candidate that optimizes running time, using the definition of neighborhood from Section 3.2. Update the value of the current candidate to minimize the objective function.

Cycle through coordinates: Iterative updates are repeated by cycling through all the coordinates in a round-robin fashion (see Section 3.3).

Convergence: The process continues until some convergence criterion (like a small enough change in the objective function or solution) is met (see Section 3.4).

3.1 Bootstrapping: Finding the Seed Kernel

In this work, we use the Polyhedral Model to *bootstrap* parameter search via coordinate descent. Bootstrapping, in this context, means finding a *Kernel Optimization Space*. To define this concept, we note that in the Polyhedral Model, typical kernels can be written as a set of polyhedra, each defined by the loop bounds. Example 3.1 illustrates this approach.

Example 3.1. In the polyhedral model, the iteration space of the loops in the matrix multiplication kernel of Figure 1 are described as a set of polyhedra defined by the loop bounds. The indices i,

j, and k form a three-dimensional space, and the bounds $0 \leq i < N$, $0 \leq j < N$, and $0 \leq k < N$ define a polyhedron (a cube) in this space defined as:

$$\mathcal{P} = \{(i, j, k) \mid 0 \leq i < N,\ 0 \leq j < N,\ 0 \leq k < N\}$$

This 3D cube represents the entire set of iterations for the three loops. Each point in this cube corresponds to a specific combination of loop indices (i, j, k), and the computation at that point updates the corresponding element $C[i][j]$.

Given the polyhedral representation of a loop, code transformations can be described as transformations of this geometric space. In other words, because the iteration space is represented as a set of integer points, transformations can be applied by multiplying these points by a transformation matrix. Example 3.2 illustrates this point.

Example 3.2. To perform a **loop interchange** on the representation seen in Example 3.1, we might swap the loops over j and k. In this case, the iteration space becomes:

$$\mathcal{P}' = \{(i, k, j) \mid 0 \leq i < N,\ 0 \leq k < N,\ 0 \leq j < N\}$$

We represent this transformation with a **permutation matrix** that swaps the second and third components of the vector. The matrix that swaps j and k is:

$$T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

This matrix has the following effect:

- The first row leaves the first coordinate i unchanged.
- The second row swaps j and k, placing k in the second position.
- The third row swaps j and k, placing j in the third position.

To apply the transformation to a point $\mathbf{p}$, we multiply the point by a matrix Θ:

$$\Theta \cdot \mathbf{p} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} i \\ j \\ k \end{pmatrix} = \begin{pmatrix} i \\ k \\ j \end{pmatrix} = \mathbf{p}'$$

This new point $\mathbf{p}'$ represents the iteration space after the loop interchange.

The notions that Examples 3.1 and 3.2 illustrate give us the material to formalize the *Kernel Optimization Space*:

Definition 3.3 (Kernel Optimization Space). Let $\mathcal{P}$ be the polyhedra-based description of a computational kernel. Let $S = \langle \Theta_1, \Theta_2, \ldots, \Theta_n \rangle$ be an ordered sequence of n transformations applied to $\mathcal{P}$; hence, leading to a new polyhedra-based description $\mathcal{P}'$. We call the sequence S a kernel optimization space relative to kernel $\mathcal{P}$.

Example 3.4. Function `tiled` (Fig. 2) shows one concrete kernel that inhabits the space that ensues from the following sequence of optimizations applied onto Figure 1: $S = \langle \texttt{interchange}_{i,j}, \texttt{tile}_j, \texttt{tile}_k \rangle$. Figure 3 shows the full space, which is formed by every possible kernel that abide by the same sequence of optimizations.

The beauty of polyhedral-based transformations is that they provide a composable technique to rewrite loops, plus the mechanisms to validate these transformations and to estimate their profit. In this regard, polytope operations give us a measure of *memory reuse*: by comparing the dimensions

of the data space (the bounds of tensors accessed by kernels) with the dimensions of the iteration space we can estimate data locality statically [7]. These observations lead to Proposition 3.5, which we state below:

Proposition 3.5. *Out-of-the-box polyhedral analyses can be effectively used to find a good kernel optimization space.*

Notice that Proposition 3.5 talks about the *kernel space* but says nothing within points within this space: this optimal point depends on hardware details. It is not simple to augment polyhedral analyses with cost models that correctly model the many architectures onto which they can be used, as already observed by previous work [56]. To achieve this goal, we resort to the ideas discussed in Section 3.2.

3.2 The Neighborhood Function

The Search Space from Definition 3.3 has a *basis*: a set of parameterizable optimizations that spans the entire space. These parameters, in the context of this paper, are natural numbers. In other words, each point of the optimization space is a choice of values for the parameters of the optimizations that define it. Example 3.6 clarifies this notion.

Example 3.6. Figure 4 shows the general description of all the kernels that form a five-dimensional optimization space, generated by the following optimizations: $S = \langle \texttt{interchange}_{i,j}, \texttt{tile}_j, \texttt{tile}_k, \texttt{parallelize}_i, \texttt{unroll}_j, \texttt{vectorize}_j \rangle$. Except for loop interchange, all these optimizations are customized via parameters. For instance, unrolling takes in one parameter: the unrolling factor, and parallelization takes in one parameter: the number of threads. In this example, we consider the following parameter sets:

- $P_0 = \texttt{parallel}_i = \{1, 2, 4, 6, 8, 10\}$
- $P_1 = \texttt{tile}_k \in \{1, 4, 8, 16\}$
- $P_2 = \texttt{tile}_j \in \{1, 4, 8, 16\}$
- $P_3 = \texttt{unroll}_j \in \{1, 8, 16, 32\}$
- $P_4 = \texttt{vectorize}_j \in \{1, 4, 8\}$

A concrete kernel k is defined by a choice of optimization parameters $(p_1, p_2, \ldots, p_n)$, where each parameter belongs to a finite set of natural numbers, e.g., $p_i \in P_i \subset \mathbb{N}$. From this observation, we define "neighborhood" as follows:

Definition 3.7 (Neighborhood). The *neighborhood* of a kernel $k = (p_1, \ldots, p_i, \ldots, p_n)$ is the set of kernels $k' = (p_1, \ldots, p'_i, \ldots, p_n)$ where either:

- $p'_i > p_i$, and for any other $p \in P_i$, if $p > p_i$, then $p \geq p'_i$;
- $p'_i < p_i$, and for any other $p \in P_i$, if $p < p_i$, then $p \leq p'_i$.

Example 3.8. Figure 4 shows the origin of the kernel optimization space, e.g., the vector $(p_0 = 1, p_1 = 1, p_2 = 1, p_3 = 1, p_4 = 1)$, and the neighborhood of this vector, which contains five new vectors.

3.3 Choosing the Next Kernel

As Definition 3.7 formalizes, a neighborhood is the set of kernels that differ from the current kernel by a change in one of its parameters, where each change represents an increment or decrement along a specific coordinate. Our implementation of coordinate descent explores the neighborhood of the current kernel candidate to identify the next kernel candidate. In this paper, we have evaluated three different techniques to choose the next kernel candidate:

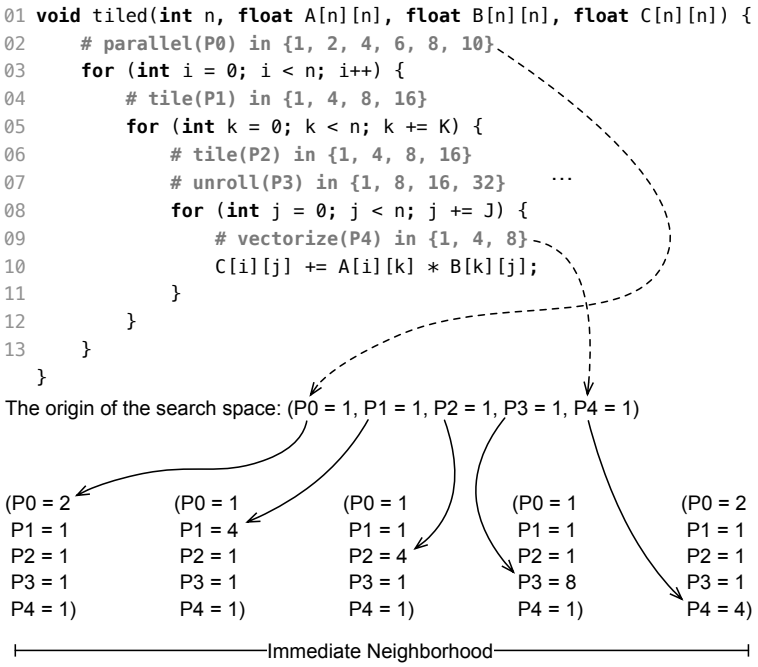

Fig. 4. Example of search space, with base and origin, plus an example of neighborhood.

(1) **Immediate Neighbor Evaluation**: In this approach, the algorithm evaluates the immediate neighbors and selects the best kernel that demonstrates a performance improvement over the current one. This method prioritizes quick progress, especially when the kernel neighborhood is large.
(2) **Expanded Neighborhoods**: The expanded neighborhoods approach broadens the search by considering not only immediate neighbors (first-degree) but also more distant candidates, known as higher-degree neighbors (second or third degree). This method allows the algorithm to reach beyond local configurations, helping it avoid getting stuck in suboptimal local minima by exploring a larger radius within the search space. While this flexibility can reveal better-performing configurations, it also introduces a trade-off, as examining higher-degree neighbors increases computational cost.
(3) **Shortest-Hop Exploration**: In this approach, once coordinate descent stops (stopping criteria are discussed in Section 3.4), the search continues using the smallest value in the list of valid optimization parameters. This technique allows exploration of points that are not present in the original list of optimization parameters, as Example 3.10 will illustrate.

These strategies offer a balance between search speed and thoroughness, allowing the coordinate search algorithm to adapt to various performance surfaces, as Example 3.9 illustrates. This example shows that coordinate descent enables the polyhedral-based optimizer to explore interactions between different optimizations that would be difficult to approximate with an exclusively static cost model.

Example 3.9. Figure 5 illustrates an example of a kernel neighborhood in a 2D space shaped by two optimizations: vectorization and unrolling. The first-fit approach explores the neighboring kernels in order. In this scenario, coordinate descent would move from the current candidate to

Kernel-3, even though Kernel-4 offers better performance. In contrast, the best-fit approach would select Kernel-4, as it explores the entire neighborhood. Lastly, an expanded search with a depth of 2 would examine nine kernels in total, ultimately choosing the best one.

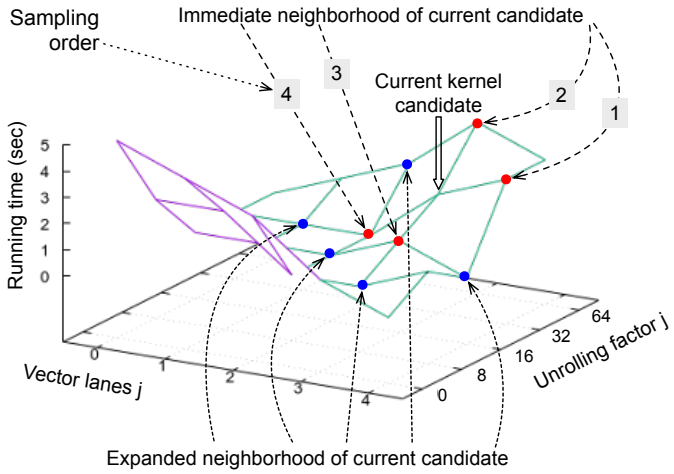

Fig. 5. Immediate and expanded neighborhoods of the kernel candidate on a 2D search space. Coordinate descent on the immediate neighborhood is faster, but has higher chance of becoming stuck in local minima.

The immediate and expanded neighborhoods discussed in Example 5 consider a predefined list of optimization parameters and do not include new parameters formed by the additive composition of old parameters. This shortcoming is addressed in the shortest-hop variation of our search, as shown in Example 3.10.

Example 3.10. Figure 6 illustrates how the shortest-hop search works. We assume an optimization, such as tiling, which initially admits five parameters: 2, 4, 8, 16, and 32. These parameters are provided by the user; however, they might not be the only valid solutions for tiling. Tiling windows of length 16, 18, or 20 could also be valid. Indeed, if any of these parameters are perfect divisors of the tiled loop, they might even be better choices, as observed by Tollenaere et al. [56]. In this example, let's assume that coordinate descent stops at tiles of size 16 (because trying tiles of size 32 does not yield faster kernels). At this stopping point, the shortest-hop search will start varying sizes by a step of two, as this is the shortest hop. The search still follows coordinate descent and evaluates tiles of length 14 and 18. If the latter yields faster kernels, it becomes the current candidate.

3.4 Stopping Criteria

Coordinate descent stops if one of the following two conditions is met:

Convergence: There is no statistical difference between the current candidate kernel K and every kernel in K's neighborhood.

Iterations: Coordinate descent runs for a fixed number N of iterations. Each iteration consists of finding a new candidate kernel K_i, which is strictly faster than the previous candidate kernel K_{i-1}.

The experiments in Section 4 use a fixed bound of $N = 100$ iterations. However, in the experiments reported in Section 4, this bound has never been achieved. Coordinate descent always stops before

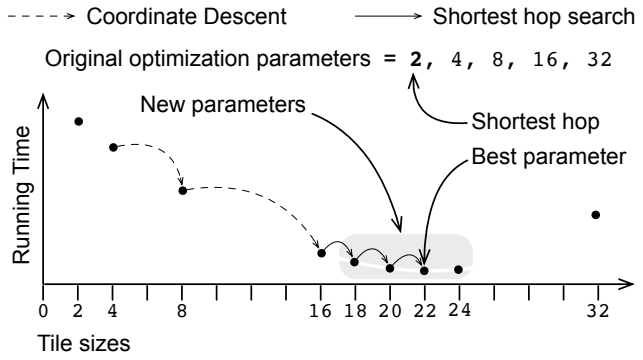

Fig. 6. Convergence via shortest-hop search.

its 100^{th} iteration, due to convergence. We determine convergence at iteration i if the current candidate kernel K_i cannot be shown to be faster than any kernel on its neighborhood via a Mann-Whitney U test. We use this non-parametric test due to the small size of samples that constitute each *population* that it compares. In our case, a population is the different running times that result from executing a kernel. Our implementation of coordinate descent runs each kernel three times. Therefore, we use the Wilcoxon rank-sum test to tests the hypothesis that the distributions of two populations, each with three samples, are the same.

4 EVALUATION

This section evaluates the following research questions:

- **RQ1:** How does our technique improve upon Pluto in terms of kernel performance?
- **RQ2:** What is the additional time overhead introduced by our approach?
- **RQ3:** How does our approach compare to other well-known tools in terms of kernel quality?
- **RQ4:** How does our approach compare to other well-known tools in terms of search time?
- **RQ5:** What is the optimal starting point for the search trajectory?
- **RQ6:** How does the choice of the next candidate improve search efficiency and kernel quality?
- **RQ7:** Can we generalize the adjustment approach to a different hardware model and a different optimization, such as thread blocking on a graphics processing unit?

Hardware: We evaluated the scheduling approaches on two architectures: x86 (Intel Xeon Max CPU 9468) and ARM-based A64FX (Fujitsu A64FX-FX700 on the Ookami Cluster). The Intel Xeon Max CPU operates at 2.6 GHz with 96 cores, and features 4.5 MiB L1d cache, 3 MiB L1i cache, 192 MiB L2 cache, and 210 MiB L3 cache. It is equipped with approximately 398 GB of RAM. The Fujitsu A64FX runs at 1.8 GHz with 48 cores and is supported by 32 GB of RAM. Each A64FX core has a 64 KB L1 cache (32 KB for data and 32 KB for instructions), and each 12-core NUMA node shares an 8 MB L2 cache. The architecture lacks an L3 cache.

Software: The ideas discussed in this paper are implemented in Pluto v0.12.0, which was the last release available at the time we performed the experiments in this section. Henceforth, we shall use **PlutoCD** when referring to the version of Pluto augmented with our implementation of coordinate descent. Pluto's standard distribution contains a collection of kernels [8]. In this section, we evaluate 11 of them. We use every kernel that we could compile (or transform) with the following tools besides Pluto: gcc v13.2.0, clang v16.0.0, Polly release 20.0.0git, TVM v0.18.0 and IOOpt [37]. Except for TVM, all the tools are given the same implementation of the kernel, in C++.

TVM, in turn, uses an implementation written in TensorIR, its internal domain-specific language. The only polyhedral optimization in Pluto that features adjustable parameters is tiling: in this case, tiling is parameterized by the length of the tiling window. Therefore, each dimension of tiling gives us a dimension in the search space that we can optimize using coordinate descent.

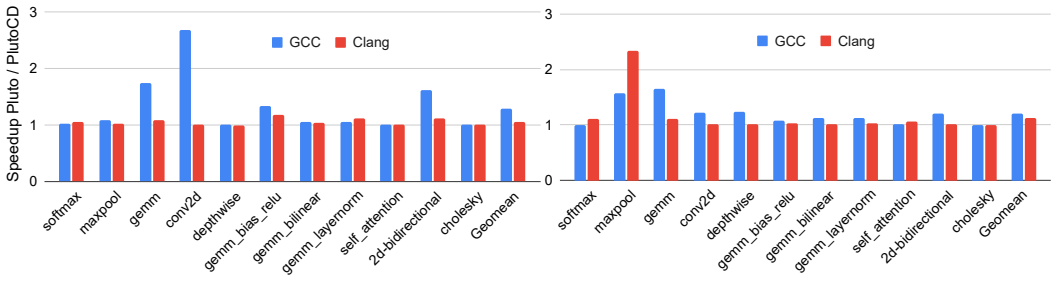

Fig. 7. Performance speedup of the proposed solution compared to Pluto on (a) x86 architecture and (b) A64FX architecture.

4.1 Speeding Up Kernels Produced by Pluto

Our first research question evaluates how much room for improvement Pluto's default choice of optimization parameters leaves. To this end, we evaluate Pluto and PlutoCD on the eleven benchmark we have. In this evaluation, we use two different compilers as code generators: gcc and clang, at the -O3 optimization level. In other words, for each kernel, we optimize it using Pluto, adjust parameters using PlutoCD, and report the ratio between the running time of Pluto's kernel over PlutoCD's.

Discussion. Figure 7 reports our speedups. Evidently, PlutoCD always produces some speedup over Pluto. This observation is expected: if PlutoCD cannot optimize any parameter of the target kernel, it defaults to Pluto's original version. Given our combinations of code generator (clang and gcc) and target architecture (x86 and A64FX), Figure 7 reports 44 different experiments. In only five cases does PlutoCD settle for the same choice of parameters that Pluto uses by default (tiling windows spanning 32 loop iterations). Nonetheless, we still observe noticeable speedups: 1.28x on gcc/x86, 1.06x on clang/x86, 1.21x on gcc/A64FX, and 1.12x on clang/A64FX (geometric mean of ratios).

4.2 Running Time Overhead of Coordinate Descent

PlutoCD adds a *search overhead* on top of Pluto. This overhead includes the time required to run coordinate descent and to sample kernels. Sampling a kernel requires running its implementation three times. The goal of this research question is to gauge the extent of this search overhead.

Discussion. The table in Figure 8 shows the search time of PlutoCD. Without any coordinate descent, we observe the following execution times for the generated code for all our eleven benchmarks (in milliseconds): x86/gcc = 184.20; x86/clang = 2486.72; ARM/gcc = 884.89; and ARM/clang = 10829. These are the execution time of the kernel produced by Pluto. Once we add coordinate descent to this pipeline — comprising the time to run coordinate descent and sample kernels combined with the execution time of the kernels — we observe (time in milliseconds): x86/gcc = 13851; x86/clang = 107224; ARM/gcc = 64731; and ARM/clang = 428901. Therefore, the slowdown that we obtain with PlutoCD instead of Pluto is: x86/gcc = 75.19x; x86/clang = 43.12x;

ARM/gcc = 73.15x; and ARM/clang = 39.60x due to the search component added to Pluto. The improvement in the kernel's execution time is: x86/gcc = 1.23x; x86/clang = 1.06x; ARM/gcc = 1.19x; and ARM/clang = 1.04x

Processor	X86 Xeon		ARM a64fx	
Kernel	**GCC (ms)**	**Clang (ms)**	**GCC (ms)**	**Clang (ms)**
softmax	3806	5762	2851	7273
maxpool	6900	117120	36654	750950
gemm	61511	566470	298080	2603529
conv2d	12159	177808	91968	972797
depthwise	180429	2179340	1377427	7978330
gemm_bias_relu	102679	654436	480187	5706932
gemm_bilinear	102908	548351	448504	4366545
gemm_layernorm	85489	1085021	457104	3492492
self_attention	35172	183203	296563	325822
2d-bidirectional	11	60	38	179
cholesky	2846	33824	19280	161574
Geomean	13851	107224	64731	428901

Fig. 8. Time taken to generate the kernel using PlutoCD (measured in milliseconds).

4.3 On the Performance of Kernels

There are various ways to produce executable implementations for the eleven benchmarks chosen in this evaluation, each representing different trade-offs between compilation time and executable quality. This section compares six approaches, each represented by a different tool: Pluto, PlutoCD, gcc -O3, Polly, IOOpt, and AutoTVM. The first five tools receive the same kernel implementation as input. In contrast, TVM receives a specification of the kernel written in its tensor expression (TVM's DSL). All optimized kernels are compiled with gcc -O3 before deployment, making gcc -O3 the baseline in this study.

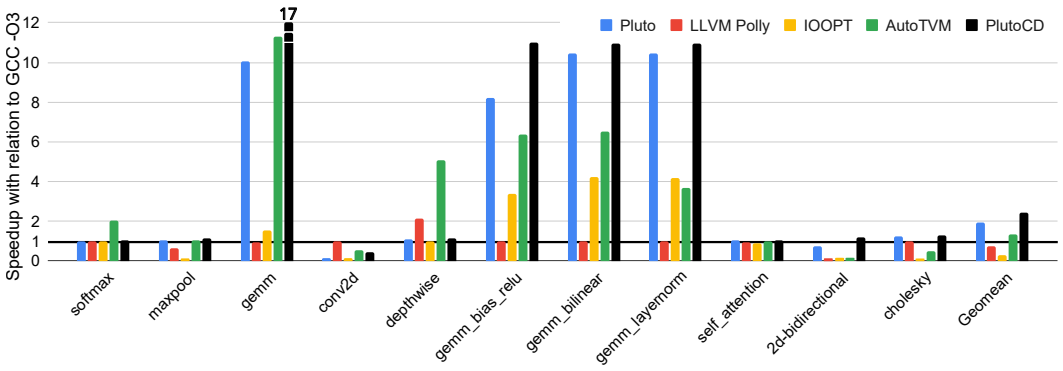

Fig. 9. Performance comparison between GCC -O3 and other tools on x86 architecture.

Discussion. Figure 9 compares the running time of executable kernels using gcc -O3 as the baseline. We observe the following average running time variations (geometric mean over ratios): x86/Pluto = 1.96x; x86/PlutoCD = 2.46x; x86/Polly = 0.74x; x86/IOOpt = 0.30x; x86/AutoTVM = 1.36x.

AutoTVM is allowed to generate and run 1,000 kernels. AutoTVM's XGBoost typically converges sooner. TVM is only effective when templates for the input kernel are available. Without templates, AutoTVM may run all 1,000 samples without finding a competitive implementation. For IOOpt, we used its online interface to obtain the shape and tiling dimensions for the input kernel. While it allows specification of cache hierarchy parameters, the chosen tiling parameters and loop ordering did not yield competitive kernels in our x86 setting. We have reported this observation to the authors.

4.4 Comparison of Search Times

There are two approaches to generate optimized versions of kernels: static code generation guided by a cost model (as done by Pluto, Polly, and IOOpt) and sampling (as done by AutoTVM and PlutoCD during the coordinate descent phase). Sampling involves executing kernels and thus might impose a significant overhead on the compilation process. This section analyzes this overhead by comparing the search times of PlutoCD and AutoTVM. AutoTVM is allowed to run 1,000 different versions of a kernel, following the same methodology used in Section 4.3.

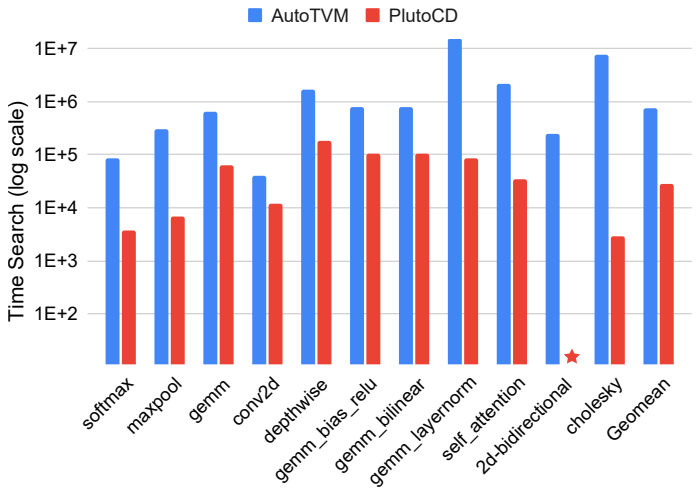

Fig. 10. Search time of AutoTVM and our tool (measured in milliseconds). The search time of PlutoCD on `2d-bidirectional` was too short to show.

Discussion. Figure 10 compares the search times of AutoTVM and PlutoCD. The figure clearly shows that PlutoCD is much faster than AutoTVM. On average, based on the geometric mean of absolute running times, PlutoCD is 24.44x faster than AutoTVM in generating all eleven executables. This advantage is further reflected in the fact that kernels produced by PlutoCD are faster than those generated by AutoTVM, as discussed in Section 4.3.

4.5 On the Starting Point of Coordinate Descent

Coordinate descent, as a line-search method, requires a *seed*, or starting point. In TVM, its application is based on the "Droplet Hypothesis" [12], a conjecture proposing that the line between the origin in the optimization space (i.e., the unoptimized kernel) and the optimal point in this space forms a convex region. Following this hypothesis, the coordinate descent should ideally begin with a configuration where each tile window has a size of one. However, Pluto provides a practical starting

point: a configuration where each tile window has a size of 32. In this section, we explore which starting point leads to the most effective implementation of PlutoCD, considering both the quality of the generated kernels and the convergence time.

Kernel	From origin		From seed	
	Search time (ms)	Kernel exec (ms)	Search time (ms)	Kernel exec (ms)
softmax	762	10	292	10
maxpool	19527	66	2527	66
gemm	193080	116	26735	112
conv2d	46130	178	29969	211
depthwise	923478	3125	650989	3129
gemm_bias_relu	416638	586	111220	587
gemm_bilinear	395848	586	79335	605
gemm_layernorm	262040	736	108420	729
self_attention	27140	425	19281	424
2d-bidirectional	98	0.24	35	0.24
cholesky	8698	260	14938	259
Geomean	35748	135	13893	137

Fig. 11. Search time and execution time (both in milliseconds), starting from the **origin** or from Pluto's **seed**.

Discussion. Figure 11 shows that beginning from Pluto's seed (tile size 32) consistently reduces convergence time across the tested kernels compared to starting from the origin (tile size one). Specifically, starting from Pluto's seed requires fewer iterations to reach an optimal configuration, suggesting that the search space around the seed provides a more favorable trajectory for coordinate descent. When initialized from the origin, the search must explore a broader parameter range before achieving comparable performance, resulting in a slower convergence rate as the algorithm takes additional steps to approach the optimal settings found near the seed. The starting point, however, appears to have little impact on the overall quality of the kernels produced by PlutoCD. Although Figure 11 shows slight variations in the "Kernel exec" columns, these differences—except in the case of `conv2d`—are likely not statistically significant. In `conv2d`, beginning from the origin of the search space (in line with the Droplet Hypothesis) produces a faster kernel, which may indicate that Pluto's seed lies outside the convex region anticipated by this hypothesis

4.6 On the Choice of Next Candidate

As explained in Section 3.3, our implementation of PlutoCD considers three different approaches to choose the next optimization parameter that coordinate descent must explore. This research question evaluates how these different strategies for selecting the next candidate in the search trajectory affect the efficiency of convergence and the quality of the final optimized kernel produced by PlutoCD

Immediate Neighbor Evaluation: The first method discussed in Section 4.6, the **best fit** approach, involves evaluating all the immediate neighbors of the current candidate. For each neighbor, we run the kernel three times and select the candidate with the best observed performance. Figures 8 and 9 already report results for this approach. Based on those results, we know that PlutoCD's default best fit approach, even when restricted to a neighborhood of distance one, yields kernels that are faster than those produced by Pluto alone. However, as the search is restricted to the immediate

neighborhood, it may fail to identify configurations that require broader exploration, potentially missing global minima due to getting stuck in local minima.

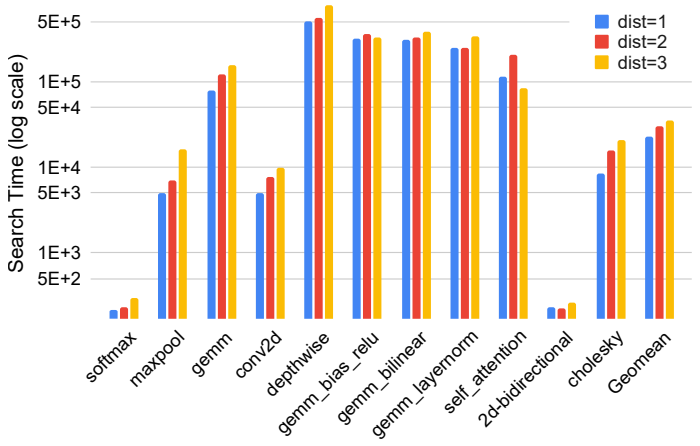

Fig. 12. Impact of neighborhood distance on search performance.

Expanded Neighborhoods: To evaluate the impact of the expanded neighborhood on PlutoCD's implementation, we evaluated neighborhoods of degree 2 and degree 3 in addition to the immediate neighborhood of degree 1 (used in Figures 8 and 9). As shown in Figure 12, each degree represents a wider radius of potential configurations from the starting point. These expanded neighborhoods could, in theory, allow PlutoCD to escape local minima. However, regardless of the neighborhood degree, all three classes converged to similarly performing candidates, corroborating the 'Droplet Hypothesis' mentioned in Section 4.5. This result suggests that increasing the degree of neighbors does not necessarily enhance performance. Instead, expanding to higher-degree neighborhoods introduces greater search-time overhead without significant improvements in kernel quality. Thus, limiting the neighborhood to degree 1 is typically more efficient, which is why this is the default configuration of PlutoCD.

Shortest-Hop Exploration: The third search approach we evaluate in this section is the shortest-hop technique described in Section 3.3 and illustrated in Example 3.10. Using this strategy, we first adjust parameters using the normal coordinate descent algorithm. Once it converges, we switch to constant variations of parameters; hence, exploiting parameter configurations that are not in the original set of predetermined configurations. Figure 13 shows how this technique fares. The shortest-hop technique yiels a marginal improvement on the running times of kernels: about 3% on top of the default PlutoCD's immediate neighborhood. However, its search time is substantially longer: about 30x. Using the shortest-hop exploration, we managed to improve on the default immediate-neighborhood method in only three benchmarks: `gemm`, `gemm_layernorm` and `self_attention`. Improvements naturally happen because the shortest-hop search explores more versions of the same kernel.

4.7 On the Generalization of the Approach

The previous techniques have evaluated the impact of parameter adjustment applied onto a single compiler optimization—tiling—when producing code for a standard CPU. This section demonstrates that this form of adjustment based on coordinate descent can be applied onto a different

Kernel	GCC -O3	Dynamic Coordinate Descent (1 degree neighbor)			Shortest-Hop Exploration		
		Search Time (ms)	Execution Time (ms)	Tile Sizes	Search Time (ms)	Execution Time (ms)	Tile Sizes
softmax	9.9	8.17E+02	10	(64,)	2.11E+04	9.8	(488,)
maxpool	103	1.23E+04	57	(4, 16)	5.29E+04	57	(16, 24)
gemm	1026	1.82E+05	105	(32,128,4)	2.17E+05	91	(256, 512, 8)
conv2d	76	1.45E+04	77	(2, 512)	3.26E+04	77	(4, 512)
depthwise	3283	7.86E+05	2511	(1,1,64,32)	1.30E+06	2500	(54,53,116,84)
gemm_bias_relu	4022	1.58E+05	735	(4, 64, 2)	3.14E+05	720	(10, 70, 8)
gemm_bilinear	3973	2.07E+05	593	(16, 64, 4)	3.04E+05	592	(116,140,110)
gemm_layernorm	3977	2.07E+05	594	(8, 64, 8)	4.61E+05	587	(264,312,252)
self_attention	1742	9.84E+04	1716	(4, 2)	3.80E+06	1619	(502, 512)
2d-bidirectional	0.25	1.10E+07	0.235	(256,)	3.96E+08	0.237	(512,)
cholesky	342	1.50E+04	260	(4, 2, 2, 2)	6.62E+04	259	(2, 2, 2, 2)

Fig. 13. Search time and execution time (both in milliseconds) for two distinct next-neighbor approaches.

optimization—*thread block tuning*—when generating code for a different architecture—the NVIDIA A100: a graphics processing unit (GPU). In CUDA, thread block tuning refers to selecting an optimal configuration of thread blocks; specifically, the number of threads assigned to each block along the X, Y, and Z dimensions [29, 53]. This choice directly impacts memory access patterns, shared memory utilization, and overall execution efficiency. Example 4.1 illustrates this optimization.

Example 4.1. Figure 14 shows part of a program that implements matrix multiplication in CUDA. Thread block tuning, in this example, involves selecting optimal values for `BLOCK_SIZE_X` and `BLOCK_SIZE_Y` to balance parallelism, memory efficiency, and hardware occupancy. In this matrix multiplication kernel, each thread computes a single output element `C[row, col]` by iterating over a row of `A` and a column of `B`. The number of threads per block (blockSize) is set as `(BLOCK_SIZE_X, BLOCK_SIZE_Y)`, while the number of blocks (`gridSize`) is computed to cover all $N \times N$ elements. Choosing blocks that are too small leads to excessive scheduling overhead, while too large blocks may waste registers and shared memory, reducing occupancy. The best values depend on GPU architecture, memory bandwidth, and workload characteristics.

Benchmarks. In this section, we evaluate the impact of post-adjustment on ten benchmarks (for the list, see Figure 15). These benchmarks were chosen to simulate the effect of compiler optimizations. The suffix "`_vanilla`" denotes the original benchmark, without any transformation. The suffix "`_unroll`" denotes the effect of loop unrolling; and the suffix "`_interchange`" denotes the effect of the interchange of the two innermost loops. Loop interchange and loop unrolling were performed manually on the original benchmark. Two benchmarks used in Figure 15 were taken from an artifact made publicly available by previous work [50]. These benchmarks simulate the application of "statement reordering", as proposed by Rawat et al.[4].

Methodology. All the results reported in this section were observed on an NVIDIA A100 graphics processing unit, where blocks of threads have access to 1,024 physical threads. We compare the relative performance of programs compiled with the following four thread adjustment plans:

[4]The "`HY`" benchmark corresponds to the "`hypterm`" routine, a component of the ExpCNS Compressible Navier-Stokes mini-application developed by the U.S. Department of Energy (DoE). The SW4 and related stencil benchmarks originate from the Seismic Wave SW4 application, designed for geodynamic simulations [22].

```
__global__ void matMul(const float *A, const float *B, float *C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        float sum = 0.0f;
        for (int k = 0; k < n; k++)
            sum += A[row * n + k] * B[k * n + col];
        C[row * n + col] = sum;
    }
}

#define N 2000
#define BLOCK_SIZE_X 16
#define BLOCK_SIZE_Y 64

int main() {
    ...
    dim3 blockSize(BLOCK_SIZE_X, BLOCK_SIZE_Y);
    size_t dim_x = (N + BLOCK_SIZE_X - 1) / BLOCK_SIZE_X;
    size_t dim_y = (N + BLOCK_SIZE_Y - 1) / BLOCK_SIZE_Y;
    dim3 gridSize(dim_x, dim_y);
    matMul<<<gridSize, blockSize>>>(dA, dB, dC, N);
    ...
}
```

The goal of thread block tuning is to determine the values of BLOCK_SIZE_X and BLOCK_SIZE_Y that maximize performance. The product BLOCK_SIZE_X * BLOCK_SIZE_Y must be less than the maximum number of threads per block.

Fig. 14. Thread block tuning consists in determining the values of BLOCK_SIZE_X and BLOCK_SIZE_Y that maximize performance.

Baseline: The default thread allocation of each benchmark. This schema assigns 1,024 threads to 1D-applications (those where threads are created along one single dimension), and 32×32 threads to 2D-applications, where threads are created along the X and Y dimensions.

Exhaustive: This approach tests every possible allocation of threads along the available dimensions, varying the allocations in steps of eight threads.

Degree-1: Adjustment via coordinate descent using a neighborhood of size 1.

Degree-2: Adjustment via coordinate descent using a neighborhood of size 2.

To evaluate the impact of post-adjustment on thread block tuning, we proceed as follows: we reset to one the number of threads in all the dimensions that are subject to change. We then apply coordinate descent onto these parameters, following the methodology explained in Section 3.3.

Discussion. Figure 15 shows that the adjustment phase, regardless of the technique used, is advantageous in almost every benchmark, albeit the degree of improvement varies considerably. The geometric average running time improvement over the baseline is 5.48%, 7.67% and 8.53% for coordinate descent with degree one, degree two and the exhaustive search, respectively. Coordinate descent delivers results that are very close to the exhaustive search. The extended neighborhood improves upon the neighborhood of degree one in four benchmarks: `vanilla_matrix_multiplication`, `sentencil_64pt_unroll`, `hy` and `redux_unroll`, with a large improvement (approximately 15%) being observed on the latter.

Convexity does not seem to be a determining factor for performance. None of the ten benchmarks seen in Figure 15 yields a convex performance surface. To demonstrate this fact, Figure 16 shows the performance surface of eight of these benchmarks. Non-convexity surfaces might cause coordinate descent stabilize at local minima. Nevertheless, although the line of coordinate descent does not traverse a convex surface, it tends to find thread configurations that are very close to the optimal performance point. To illustrate this observation, Figure 17 shows the search space of the vanilla

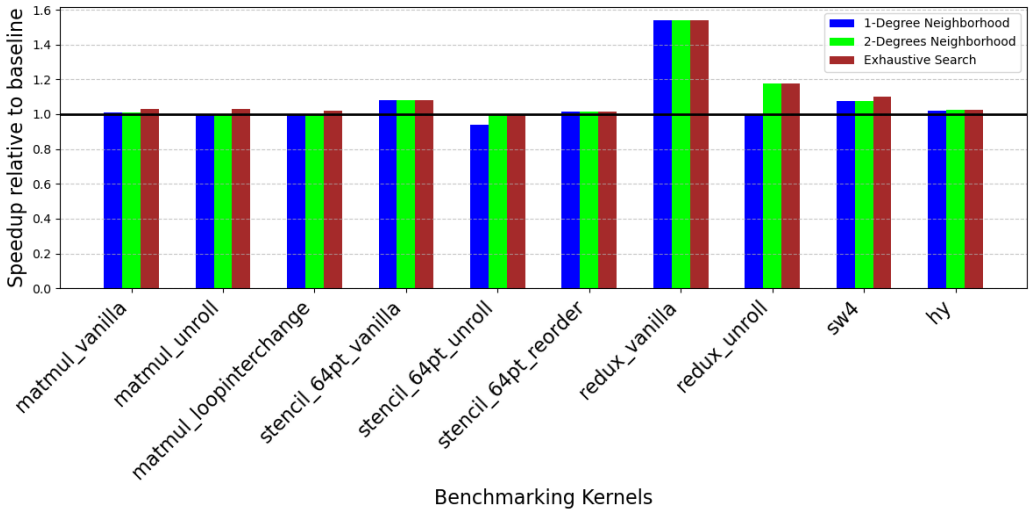

Fig. 15. Impact of thread block tuning on the performance of typical Cuda benchmarks. Each number is the average of three results.

matrix multiplication kernel. In this example, each one of the four thread-configuration approaches ends with a different configuration. Yet, as the figure emphasizes, coordinate descent, although not zeroing on the optimal configuration, is very close to it.

5 RELATED WORK

This paper unifies two code generation techniques: auto-tuning and polyhedral-based compilation. Both share the fundamental goal of producing highly efficient code that maximizes performance by exploiting the underlying hardware's capabilities. Additionally, both approaches focus on loop optimizations, such as tiling, unrolling, and fusion, and target performance-sensitive domains such as scientific computing, image processing, and deep learning. The key difference between these approaches lies in how they explore the optimization space and make decisions. Figure 18 highlights these differences.

Auto-tuning, as seen in tools like MetaTune [51], AutoTVM [16], Tiramisu [6], Halide [45], OpenTuner [4], CLTune [35], Orio [28], ATF [49], MAQAO [19] and Ansor [64], is a feedback-driven approach. It generates multiple versions of code with different optimization configurations, runs these versions on the target hardware, and collects performance data, such as execution time and memory usage. On the other hand, polyhedral-based code generation, as applied in tools like Pluto [9, 11], Polly [25, 33], PoCC [39], Polygeist [34], GRAPHITE [38], PolySA [18], PPCG [58], and CHiLL [14], is a theoretical model-driven approach: optimizations are guided by a cost model rather than by runtime feedback.

One primary advantage of auto-tuning is its ability to tailor optimizations to specific hardware architectures and workloads by empirically testing multiple code versions. This adaptability allows auto-tuners to optimize code for diverse architectures, from CPUs to GPUs to FPGAs, without relying on theoretical models. However, this trial-and-error process is computationally expensive, as running multiple code versions and gathering performance data is time-consuming.

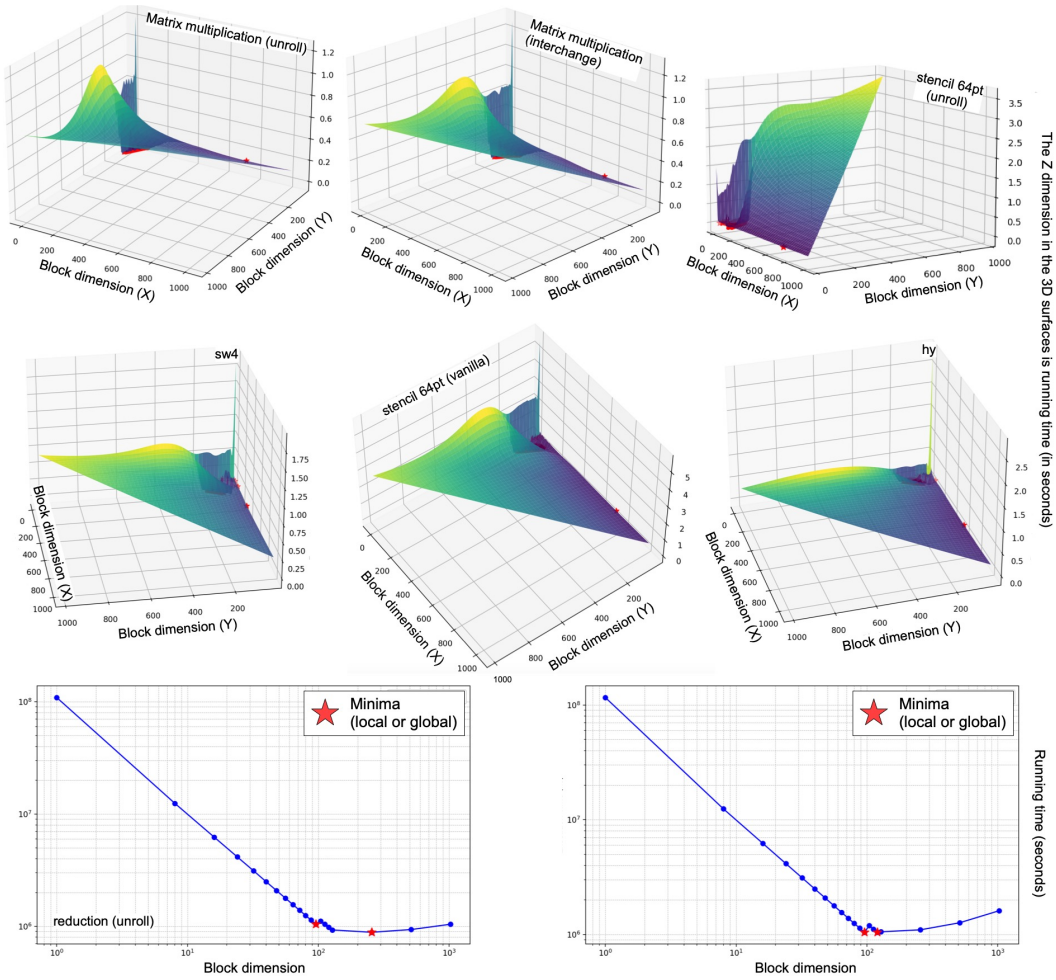

Fig. 16. The search space of thread blocks for seven benchmarks evaluated in this section. Each point is the average of three runs. Red stars mark points of minima. Every surface has at least two such points.

Bridging the Gap. This paper proposes a simple technique to unify polyhedral and search-based code generation. However, previous works have already combined these techniques. One of the earliest is Pouchet et al. [40]'s iterative polyhedral compiler, which uses the polyhedral model to prune the auto-tuner's search space. More recently, Tollenaere et al. [56] have used similar techniques to speed up Ansor's empirical search, avoiding evaluation of kernel versions that are unlikely to yield good performance.

We believe that using coordinate descent to adjust the parameters of polyhedral-based optimizations is an original contribution of this paper. However, coordinate descent in autotuning is not new; the droplet search algorithm was designed to speed up AutoTVM's search time [12]. Variants of this algorithm were subsequently applied to Ansor [32] and TVM's MetaScheduler [13]. The key difference in this work is that previous uses of coordinate descent were in tandem with autotuners, with initial spaces chosen via human intervention [12] or other autotuning algorithms

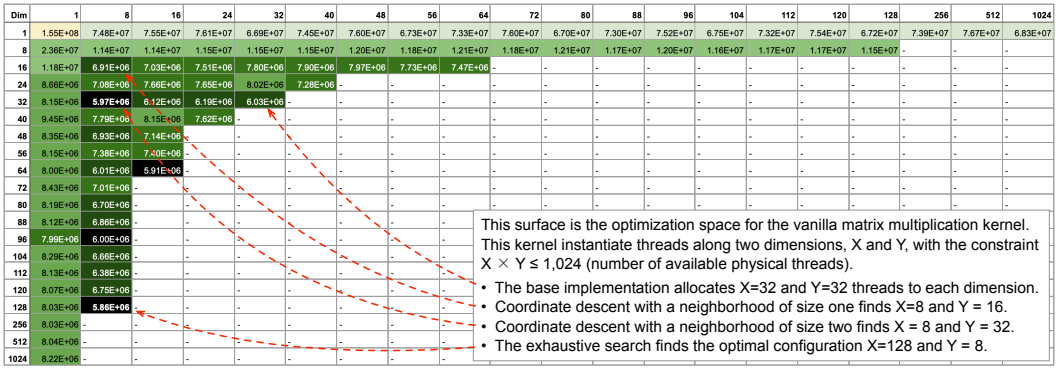

Fig. 17. The search space of the vanilla matrix multiplication kernel, showing the thread configurations found by different techniques.

	Polyhedral Compilation	Search-Based Autotuning
Modus Operandi	Analyze & Compile	(Compile → Sample → Adjust)*
Parameter adjustment	Cost model	Sampling
Tools	Pluto Polly PoCC	MetaTune AutoTVM Tiramisu
	Polygeist GRAPHITE	Halide OpenTuner CLTune
	PolySA PPCG CHiLL	Orio ATF MAQAO Ansor

Fig. 18. High-level comparison between polyhedral compilation and compiler autotuners.

like evolutionary search [13]. This paper exclusively uses static analyses based on the polytope model to determine the search space for coordinate descent.

6 CONCLUSION

This paper has introduced a methodology to adjust the optimization parameters used by polyhedral-based compilers, making them more competitive with the autotuners that have gained popularity in recent years. The approach proposed in this paper uses coordinate-descent to fine tune the parameters of static compiler optimization. The strength of this method lies in harmonizing two distinct frameworks – the polytope model and dynamic autotuning – combining them in a complementary and powerful way. On one hand, the polyhedral-based compiler provides coordinate descent with a feasible optimization space to explore. On the other, the feedback-driven adjustment phase makes the compiler more attuned to the finer details of the target hardware. Without the optimization space generated by the polyhedral compiler, coordinate descent would lack direction as it would have no clear space to navigate. Conversely, without the refinement phase of coordinate descent, effectively using the polyhedral-based compiler would be challenging, as it would not have the necessary adjustments to align its static optimization parameters with the complexities of the target architecture—a task too intricate to be captured by a general cost model.

Software. An artifact containing all the tools necessary to reproduce the experiments in Section 4 is available at https://github.com/xintin/kelpie.

REFERENCES

[1] Khaled Abdelaal and Martin Kong. 2021. Tile size selection of affine programs for GPGPUs using polyhedral cross-compilation. In *Supercomputing*. ACM, New York, USA, 13–26.

[2] S. Amarasinghe and M. Lam. 1993. Communication Optimization and Code Generation for Distributed Memory Machines. In *PLDI*. ACM Press, Albuquerque, N.M., 126–138.

[3] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: an extensible framework for program autotuning. In *PACT* (Edmonton, AB, Canada). Association for Computing Machinery, New York, NY, USA, 303–316. https://doi.org/10.1145/2628071.2628092

[4] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. Opentuner: An extensible framework for program autotuning. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 303–316.

[5] Riyadh Baghdadi, Abdelkader Nadir Debbagh, Kamel Abdous, Fatima Zohra Benhamida, Alex Renda, Jonathan Elliott Frankle, Michael Carbin, and Saman Amarasinghe. 2020. TIRAMISU: A polyhedral compiler for dense and sparse deep learning. *arXiv preprint arXiv:2005.04091* (2020).

[6] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 193–205.

[7] Mohamed-Walid Benabderrahmane, Louis-Noël Pouchet, Albert Cohen, and Cédric Bastoul. 2010. The polyhedral model is more widely applicable than you think. In *Compiler Construction* (Paphos, Cyprus) *(CC'10/ETAPS'10)*. Springer-Verlag, Berlin, Heidelberg, 283–303. https://doi.org/10.1007/978-3-642-11970-5_16

[8] Uday Bondhugula. 2024. Pluto High-Performance Kernels. https://github.com/bondhugula/pluto/tree/master/examples.

[9] Uday Bondhugula, Muthu Baskaran, Sriram Krishnamoorthy, Jagannathan Ramanujam, Atanas Rountev, and Ponnuswamy Sadayappan. 2008. Automatic transformations for communication-minimized parallelization and locality optimization in the polyhedral model. In *Compiler Construction*. Springer, 132–146.

[10] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A practical automatic polyhedral parallelizer and locality optimizer. In *PLDI* (Tucson, AZ, USA) *(PLDI '08)*. Association for Computing Machinery, New York, NY, USA, 101–113. https://doi.org/10.1145/1375581.1375595

[11] Uday Bondhugula, Albert Hartono, Jagannathan Ramanujam, and Ponnuswamy Sadayappan. 2008. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 101–113.

[12] Michael Canesche, Vanderson M. Rosario, Edson Borin, and Fernando Magno Quintão Pereira. 2024. The Droplet Search Algorithm for Kernel Scheduling. , 25 pages. https://doi.org/10.1145/3650109 Just Accepted.

[13] Michael Canesche, Gaurav Verma, and Fernando Magno Quintao Pereira. 2024. Explore as a Storm, Exploit as a Raindrop: On the Benefit of Fine-Tuning Kernel Schedulers with Coordinate Descent. arXiv:2406.20037 [cs.LG] https://arxiv.org/abs/2406.20037

[14] Chun Chen, Jacqueline Chame, and Mary Hall. 2008. *CHiLL: A framework for composing high-level loop transformations*. Technical Report. Citeseer.

[15] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *OSDI*. USENIX, Berkeley, USA, 578–594.

[16] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[17] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759* (2014).

[18] Jason Cong and Jie Wang. 2018. PolySA: Polyhedral-based systolic array auto-compilation. In *ICCAD*. IEEE, New York, US, 1–8.

[19] Lamia Djoudi, Denis Barthou, Patrick Carribault, Christophe Lemuet, Jean-Thomas Acquaviva, William Jalby, et al. 2005. Maqao: Modular assembler quality analyzer and optimizer for itanium 2. In *The 4th Workshop on EPIC architectures and compiler technology, San Jose*, Vol. 200.

[20] Paul Feautrier. 1992. Some Efficient Solutions to the Affine Scheduling Problem. Part I. One-dimensional Time. *International Journal of Parallel Programming* 21, 5 (1992), 313–347.

[21] Paul Feautrier. 1992. Some Efficient Solutions to the Affine Scheduling Problem. Part II. Multidimensional Time. *International Journal of Parallel Programming* 21, 6 (1992), 389–420.

[22] Center for Exascale Simulation of Combustion in Turbulence. [n. d.]. SW4 2014. Seismic Wave Modelling (SW4) - Computational Infrastructure for Geodynamics. https://geodynamics.org/cig/software/sw4. (2014).

[23] J. A. B. Fortes and D. Moldovan. 1984. Data Broadcasting in Linearly Scheduled Array Processors. In *Symposium on Computer Architecture*. 224–231.

[24] Franz Franchetti, Tze Meng Low, Doru Thom Popovici, Richard M Veras, Daniele G Spampinato, Jeremy R Johnson, Markus Püschel, James C Hoe, and José MF Moura. 2018. SPIRAL: Extreme performance portability. *Proc. IEEE* 106, 11 (2018), 1935–1968.

[25] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. 2012. Polly—performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters* 22, 04 (2012), 1250010.

[26] Tobias Grosser, Armin Größlinger, and Christian Lengauer. 2012. Polly - Performing Polyhedral Optimizations on a Low-Level Intermediate Representation. *Parallel Process. Lett.* 22, 4 (2012), 23 pages. https://doi.org/10.1142/S0129626412500107

[27] Rajiv Gupta, Santosh Pande, Kleanthis Psarris, and Vivek Sarkar. 1999. Compilation techniques for parallel systems. *Parallel Comput.* 25, 13-14 (1999), 1741–1783.

[28] Albert Hartono, Boyana Norris, and Ponnuswamy Sadayappan. 2009. Annotation-based empirical performance tuning using Orio. In *2009 IEEE International Symposium on Parallel & Distributed Processing*. IEEE, 1–11.

[29] Pieter Hijma, Stijn Heldens, Alessio Sclocco, Ben van Werkhoven, and Henri E. Bal. 2023. Optimization Techniques for GPU Programming. *ACM Comput. Surv.* 55, 11, Article 239 (March 2023), 81 pages. https://doi.org/10.1145/3570638

[30] F. Irigoin and R. Triolet. 1988. Supernode Partitioning. In *POPL*. ACM, 319–328.

[31] M. S. Lam. 1989. *A Systolic Array Optimizing Computer*. Kluwer Academic (Springer). https://doi.org/10.1007/978-1-4613-1705-0 ISBN:-13: 978-14612-8961-6.

[32] Chendi Li, Yufan Xu, Sina Mahdipour Saravani, and Ponnuswamy Sadayappan. 2024. Accelerated Auto-Tuning of GPU Kernels for Tensor Computations. In *Supercomputing* (Kyoto, Japan) *(ICS '24)*. Association for Computing Machinery, New York, NY, USA, 549–561. https://doi.org/10.1145/3650200.3656626

[33] LLVM. [n. d.]. Polly - LLVM Project. https://polly.llvm.org/. Accessed: 2024-10-31.

[34] William S Moses, Lorenzo Chelini, Ruizhe Zhao, and Oleksandr Zinenko. 2021. Polygeist: Raising C to polyhedral MLIR. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 45–59.

[35] Cedric Nugteren and Valeriu Codreanu. 2015. CLTune: A generic auto-tuner for OpenCL kernels. In *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*. IEEE, 195–202.

[36] Tiago Daniel Costa Oliveira. 2023. *Analysis and Comparison of Automatic Parallelization Tools*. Master's thesis. Instituto Politecnico do Cavado e do Ave (Portugal).

[37] Auguste Olivry, Guillaume Iooss, Nicolas Tollenaere, Atanas Rountev, P. Sadayappan, and Fabrice Rastello. 2021. IOOpt: automatic derivation of I/O complexity bounds for affine programs. In *PLDI* (Virtual, Canada). Association for Computing Machinery, New York, NY, USA, 1187–1202. https://doi.org/10.1145/3453483.3454103

[38] Sebastian Pop, Albert Cohen, Cédric Bastoul, Sylvain Girbal, Georges-André Silber, and Nicolas Vasilache. 2006. GRAPHITE: Polyhedral analyses and optimizations for GCC. In *proceedings of the 2006 GCC developers summit*, Vol. 6. Citeseer, 90–91.

[39] Louis-Noel Pouchet. [n. d.]. PoCC - The Polyhedral Compiler Collection. https://sourceforge.net/projects/pocc/files/?source=navbar. Accessed: 2024-10-31.

[40] Louis-Noël Pouchet, Cédric Bastoul, Albert Cohen, and John Cavazos. 2008. Iterative optimization in the polyhedral model: part ii, multidimensional time. In *PLDI* (Tucson, AZ, USA). Association for Computing Machinery, New York, NY, USA, 90–100. https://doi.org/10.1145/1375581.1375594

[41] Louis-Noël Pouchet, Uday Bondhugula, Cédric Bastoul, Albert Cohen, R Ramanujam, and P Sadayappan. 2009. *Hybrid iterative and model-driven optimization in the polyhedral model*. Ph. D. Dissertation. INRIA.

[42] W. Pugh. 1992. A practical algorithm for exact array dependence analysis. *Commun. ACM* 35, 8 (1992), 102–114. https://doi.org/10.1145/135226.135233

[43] P. Quinton and V. Van Dongen. 1989. The Mapping of Linear Recurrence Equations on Regular Arrays. *Journal of VLSI Signal Processing* 1, 2 (1989), 95–113.

[44] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *SIGPLAN Not.* 48, 6 (jun 2013), 519–530. https://doi.org/10.1145/2499370.2462176

[45] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices* 48, 6 (2013), 519–530.

[46] S. V. Rajopadhye. 1986. *Synthesis, Optimization and Verification of Systolic Architectures*. Ph. D. Dissertation. University of Utah, Salt Lake City, Utah 84112.

[47] S. V. Rajopadhye, S. Purushothaman, and R. M. Fujimoto. 1986. On Synthesizing Systolic Arrays from Recurrence Equations with Linear Dependencies. In *Foundations of Software Technology and Theoretical Computer Science*. Springer Verlag, LNCS 241, New Delhi, India, 488–503.

[48] J. Ramanujam and P. Sadayappan. 1990. Nested Loop Tiling for Distributed Memory Multiprocessors. In *Distributed Memory Computer Conference, V.* Charleston, 1088–1096.

[49] Ari Rasch and Sergei Gorlatch. 2018. ATF: A generic auto-tuning framework. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*. 3–4.

[50] Prashant Singh Rawat, Fabrice Rastello, Aravind Sukumaran-Rajam, Louis-Noël Pouchet, Atanas Rountev, and P. Sadayappan. 2018. Register optimizations for stencils on GPUs. In *PPoPP* (Vienna, Austria). Association for Computing Machinery, New York, NY, USA, 168–182. https://doi.org/10.1145/3178487.3178500

[51] Jaehun Ryu and Hyojin Sung. 2021. Metatune: Meta-learning based cost model for fast and efficient auto-tuning frameworks. *arXiv preprint arXiv:2102.04199* (2021).

[52] R. Schreiber and J. Dongarra. 1990. *Automatic blocking of nested loops.* Technical Report 90.38. RIACS, NASA Ames Research Center.

[53] Ankit Sethia and Scott Mahlke. 2014. Equalizer: Dynamic Tuning of GPU Resources for Efficient Execution. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, United Kingdom) *(MICRO-47)*. IEEE Computer Society, USA, 647–658. https://doi.org/10.1109/MICRO.2014.16

[54] Arun Thangamani, Vincent Loechner, and Stéphane Genaud. 2024. A Survey of General-purpose Polyhedral Compilers. *ACM Trans. Archit. Code Optim.* (June 2024). https://doi.org/10.1145/3674735 Just Accepted.

[55] Nicolas Tollenaere. 2022. *Decoupling the optimization space of tensor computation for a better understanding of performance on Intel CPU.* Ph. D. Dissertation. Université Grenoble Alpes [2020-....].

[56] Nicolas Tollenaere, Guillaume Iooss, Stéphane Pouget, Hugo Brunie, Christophe Guillon, Albert Cohen, P. Sadayappan, and Fabrice Rastello. 2023. Autotuning Convolutions Is Easier Than You Think. *ACM Trans. Archit. Code Optim.* 20, 2, Article 20 (March 2023), 24 pages. https://doi.org/10.1145/3570641

[57] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730* (2018).

[58] Sven Verdoolaege, Juan Carlos Juega, Albert Cohen, Jose Ignacio Gomez, Christian Tenllado, and Francky Catthoor. 2013. Polyhedral parallel code generation for CUDA. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 4 (2013), 1–23.

[59] R Clint Whaley, Antoine Petitet, and Jack J Dongarra. 2001. Automated empirical optimizations of software and the ATLAS project. *Parallel computing* 27, 1-2 (2001), 3–35.

[60] M. Wolf and M. Lam. 1991. Loop transformation theory and an algorithm to maximize parallelism. *Transactions on Parallel and Distributed Systems* 2, 4 (Oct 1991), 452–471.

[61] M. J. Wolfe. 1987. Iteration space tiling for memory hierarchies. *Parallel Processing for Scientific Computing (SIAM)* (1987), 357–361.

[62] Yuanrui Zhang. 2011. *Cache-aware application parallelization and optimization for multicores.* The Pennsylvania State University.

[63] Yifan Zhao, Hashim Sharif, Vikram Adve, and Sasa Misailovic. 2024. Felix: Optimizing Tensor Programs with Gradient Descent. In *ASPLOS* (La Jolla, CA, USA). Association for Computing Machinery, New York, NY, USA, 367–381. https://doi.org/10.1145/3620666.3651348

[64] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 863–879.

[65] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *OSDI*. USENIX Association, USA, Article 49, 17 pages.

[66] Oleksandr Zinenko, Sven Verdoolaege, Chandan Reddy, Jun Shirako, Tobias Grosser, Vivek Sarkar, and Albert Cohen. 2017. *Unified polyhedral modeling of temporal and spatial locality.* Ph. D. Dissertation. Inria Paris.